

CE 356: Formal Spec and Verification

Term Project — Midterm Progress Report

Specification and Verification of Dijkstra's Self-Stabilization Algorithm in TLA+

Joshua | Spring 2026

1. Algorithm Overview

Dijkstra's self-stabilization algorithm addresses a fundamental challenge in distributed systems: how can a system recover from an arbitrary broken state using only local information, with no central coordinator? The algorithm demonstrates that a ring of finite-state processes can always converge to a legitimate state regardless of how corrupted the initial state is through simple local rules alone.

2. System Structure

The system consists of $N+1$ processes numbered 0 through N , arranged in a ring. Each process holds a single integer value. Each process can only observe its immediate left neighbor's value. Importantly, there is no global shared memory or message passing.

A scheduler, called a "demon" in the paper, picks one process at a time to execute. The demon is assumed to be fair, meaning that every process gets selected infinitely often.

3. Privilege Rules

A process is called "privileged" if it holds the token, meaning it is currently allowed to take an action. The privilege rules are:

- Process 0 (special): privileged if its value equals its right neighbor's value (process N).
- All other processes (1 through N): privileged if their value differs from their left neighbor's value.

4. Action Rules

When a privileged process is selected by the demon, it executes the following action:

- Process 0: increments its value by 1 modulo K .
- All other processes: copy their left neighbor's value.

A non-privileged process takes no action when selected — its state remains unchanged.

5. Legitimate State and Correctness Condition

A state is legitimate if and only if:

- Exactly one process is privileged.
- All successor states are also legitimate.

The correctness condition (self-stabilization) is: starting from any arbitrary state, the system must reach a legitimate state in a finite number of steps.

6. Concrete Example (N=3, K=4)

To illustrate the algorithm, consider a system with 4 processes (0–3) and counter modulo K=4. The following trace shows the system starting from a broken state with multiple privileged processes and converging to a legitimate state.

Note: Left neighbor is the number smaller or largest if process 0

Initial (broken) state:

Process:	0	1	2	3
Value:	2	0	1	2

All four processes are privileged — a broken state with too many tokens. The demon picks process 1; it copies its left neighbor (process 0), setting its value to 2.

Process:	0	1	2	3
Value:	2	2	1	2

(3 privileged)

The demon picks process 2; it copies process 1, setting its value to 2.

Process:	0	1	2	3
Value:	2	2	2	2

(1 privileged — process 0)

Now only process 0 is privileged (its value equals process 3's value). This is a legitimate state. When process 0 fires, it increments to 3, making process 1 privileged. The token then rotates rightward around the ring.

7. Why It Self-Stabilizes: The Coloring Argument

Dijkstra proves self-stabilization using a coloring argument. When process 0 fires for the first time (from any arbitrary starting state), its new state is colored "blue". All prior states are "white". From this point:

- Any state created by process 0 is blue.
- Any state copied from a blue state is also blue.
- Blue states propagate rightward around the ring through copying.

Since $K \geq N$, process 0 can generate N distinct blue states before any white state "catches up" from the right. Once all N+1 process values are blue and equal, the system has reached a legitimate state. This guarantees convergence in a finite number of steps from any starting configuration.

8. Key Constraint: Why $K \geq N$

The requirement $K \geq N$ is critical. If $K < N$, the modular wrapping of process 0's counter could produce a value that matches a white (stale) value still circulating in the ring. This would break the coloring argument and the system might fail to stabilize. Dijkstra explicitly shows that for $K < N$, counterexamples to self-stabilization exist.

9. Planned Extension

The project guidelines encourage modifying the algorithm while preserving self-stabilization. One natural extension is to explore whether the algorithm remains self-stabilizing under a different privilege rule for process 0. For example, I may use a different increment step or a different comparison condition.

A potential modification: instead of incrementing by 1, process 0 increments by 2 (mod K). The key questions are:

- Does the coloring argument still hold?
- Does K need to be larger to compensate?
- Can TLC find a counterexample for small K, confirming the constraint needs adjusting?

Now that the base specification has been verified with TLC, the extension will be implemented by modifying the Act0 rule and re-running TLC to check whether self-stabilization still holds. The TLA+ spec is already parameterized on N and K, making it straightforward to experiment with modifications.

10. TLC Model Checking Results

The TLA+ specification was model checked using TLC with constants $N=3$ and $K=4$. The behavior spec used was FairSpec (which includes weak fairness on Next, capturing the fair demon assumption). The property checked was $\text{SelfStabilization} == \langle \rangle \text{Legitimate}$.

Results: `Model checking completed. No error has been found.`

TLC completed in approximately 2.5 seconds. Key statistics from the model checking run:

- 256 distinct initial states
 - Exactly $K^{(N+1)} = 4^4 = 256$, confirming TLC explored every possible starting configuration.
- 896 total states generated
 - TLC explored all reachable transitions between states, not just starting configurations.
- No deadlocks detected
 - This confirms that the system never reaches a state with zero privileged processes, consistent with the theoretical guarantee that at least one process is always privileged.

These results confirm that the TLA+ specification correctly captures Dijkstra's algorithm and that the SelfStabilization property holds for all 256 possible starting states with $N=3$ and $K=4$. The next step is to modify the algorithm and re-run TLC to verify the extension.